Data Curation and Model Souping: Unlocking Qwen3-0.6B for MS MARCO Passage Ranking

Shuguang Han
Xianyu of Alibaba
shuguang.sh@alibaba-inc.com

Abstract

This report studies passage retrieval and re-ranking on the MS MARCO benchmark using lightweight Qwen3-0.6B models. Across three experimental settings — BM25-then-reranking task, corpus-wide dense retrieval task, and dense retrieval then re-ranking task — the results demonstrate that competitive ranking performance can be achieved without complex architectural modifications, custom loss functions, or cross-encoder distillation. Instead, substantial gains stem from data-centric strategies: corpus-targeted continued pre-training, rigorous data curation, hard-negative mining, training–inference distribution alignment, and weight merging via Model Soup. In particular, Model Soup delivers marked performance gains while preserving the inference efficiency of a single model. These findings highlight how data curation and weight merging can push sub-billion-parameter models to state-of-the-art performance.

1 Introduction

MS MARCO passage ranking is an information retrieval task built on top of the MS MARCO dataset [1]. This dataset contains 1 million real-world Bing search queries (each framed as a question) and 8.8 million candidate passages. For each query, human annotators have labeled zero or more relevant passages. Prior work has primarily focused on two benchmark tasks: passage re-ranking and passage full ranking.

- **Passage Re-ranking**: For each query, systems are provided with the top 1,000 candidate passages retrieved by BM25. The goal is to re-rank these candidates based on their relevance to the query—specifically, the likelihood of a passage answering the question.
- **Passage Full Ranking**: While re-ranking performance is inherently bounded by the recall of the top 1,000 BM25 results, the full ranking task requires retrieving and ranking relevant passages directly from the entire corpus of 8.8 million documents. In practice, this setup typically requires developing both a first-stage retrieval model and a second-stage re-ranking model.

For MS MARCO retrieval and re-ranking, BERT-family models have long served as the dominant paradigm. Most existing studies build upon BERT [2], RoBERTa [3], ELECTRA [4], or ERNIE [12, 27], with some—such as SimLM [21] and CoT-MAE [26]—further exploring specialized pre-training objectives. More recently, with the rapid advancement of Large Language Models (LLMs), recent studies have begun examining the utility of state-of-the-art generative language models such as T5 and LLaMA for superior ranking performance [5–7].

In this report, I follow this line of work by leveraging LLMs to improve ranking quality. Specifically, in collaboration with Claude Code (powered by Claude Opus 4.7), I explore

Preprint.

effective strategies for utilizing the Qwen3-0.6B model [7] in both passage retrieval and re-ranking. I find that even without special pre-training, distillation from larger teacher models, loss function modifications, or architectural changes, rankers and retrievers built on Qwen3-0.6B can achieve state-of-the-art performance solely through deliberate training data curation and model merging. In particular, the Model Soup algorithm [17]—which averages weights across multiple fine-tuned model checkpoints—proves to be an exceptionally effective strategy for boosting model capability.

During this research, I also noticed that performance metrics reported across existing studies are not directly comparable: while most prior works omit passage titles, a subset incorporates them. In my experiments, appending passage titles consistently yields an improvement of 2–3 percentage points (pp) in MRR@10 for retrieval and 1–2 pp for re-ranking. This discrepancy aligns with observations previously noted by Lassance and Clinchant [23]. Consequently, to ensure fair and rigorous comparisons, all baseline metrics from prior literature in this report are systematically reproduced using their open-source checkpoints.

Another noteworthy aspect of this work is that throughout the entire project, I did not write a single line of code myself. In fact, given the sheer volume of code generated, comprehensively inspecting its correctness by hand was practically impossible. Claude Code provided actionable guidance during execution and demonstrated strong capabilities in diagnosing model performance issues. For instance, upon identifying the discrepancy between published results and my reproduced results, Claude Code deduced the inconsistent usage of title content and autonomously designed an experiment suite to validate the hypothesis. Nevertheless, the high-level intuition and direction behind each experiment still relied on my input, as Claude Code tended to stick to more generic solutions.

2 Methodology

In this technical report, I study three distinct experimental setups: (1) the BM25-then-Reranking pipeline, which re-ranks the top 1,000 candidate passages retrieved by BM25; (2) the Dense Retrieval task, which retrieves the most relevant passages directly from the entire corpus; and (3) the DenseRetrieval-then-Reranking pipeline, which re-ranks the top 1,000 passages retrieved by an initial dense retriever.

2.1 Backbone Models

All of my experiments are conducted by fine-tuning two Qwen3-0.6B based models [7]: Qwen3-0.6B-Embedding for retrieval and Qwen3-0.6B-Reranker for re-ranking. According to their technical report [7], both models are developed via multi-stage training pipelines built on top of the Qwen3 foundation model, implementing dual-encoder and cross-encoder architectures, respectively.

More specifically, both models are initialized directly from the pre-trained Qwen3-0.6B language model, thereby inheriting strong multilingual, long-context capabilities. The Embedding model adopts a dual-encoder architecture optimized through contrastive pre-training, supervised fine-tuning on high-quality text pairs, and model merging to maximize generalization. It extracts semantic representations from the final end-of-sequence token and supports Matryoshka Representation Learning (MRL) for flexible vector dimensions. The Reranker model employs a cross-encoder that jointly processes query-document pairs via deep cross-attention layers. It is trained with supervised fine-tuning on labeled relevance pairs to predict query-passage relevance.

Due to GPU resource constraints, I restrict my focus in this report to sub-1B models, leaving larger-sized models such as Qwen3-Embedding-4B/8B and Qwen3-Reranker-4B/8B for future exploration. Nevertheless, I expect that ranking performance could be scaled further with larger parameter sizes, as observed in the original Qwen3 Embedding report [7].

2.2 Data Preparation

Datasets. I evaluate on the MS MARCO passage ranking benchmark [1], which contains 501,233 queries in the training set and 6,800 queries in the development set, evaluated over a corpus of more than 8.8 million passages. In this dataset, each query is paired with one or more human-labeled relevant passages. The official benchmark provides the top 1,000 candidate passages per query retrieved by an un-tuned BM25 system (which can be substantially improved with hyperparameter tuning [9]).

Training Data Construction. For each of the three tasks mentioned above, I construct dedicated training datasets as follows:

The BM25-then-Reranking task: The dataset is constructed in two steps. First, I sample N queries from the training partition. Next, for each sampled query, I take the top 1,000 passages retrieved by the official BM25 model, keep all labeled relevant passages as positive samples, and randomly sample M irrelevant passages as negatives. Here, N (query count) and M (negative samples per query) are the primary parameters controlling the training scale. A representative data record is formatted as follows:

```
[system]
Judge whether the Document meets the requirements based on the Query
and the Instruct provided. Note that the answer can only be "yes"
or "no".

[user]
<Instruct>: Given a web search query, retrieve relevant passages
that answer the query
<Query>: what kind of organism is a black damsel
<Document>: Body of the organism is made up of a single cell. The
body of mutlicellular organism is made up of numerous cells. Body
organization is simple. Organization is complex ...

[assistant]

no
```

The Retrieval task: The goal is to retrieve relevant passages directly from the full 8.8-million-passage collection. To maintain low inference latency, the dual-encoder architecture is preferred over the cross-encoder for corpus-wide similarity search. The data format for training and inference is illustrated below. To boost retrieval accuracy, my focus—consistent with prior studies [19–22]—primarily centers on effective hard negative mining strategies. Specifically, I follow conventional practice by sampling hard negatives from candidates ranked between positions 20 and 200 by an existing retrieval model or cross-encoder reranker.

```
Query side:   Instruct: Given a web search query, retrieve relevant
              passages that answer the query
              Query: what kind of organism is a black damsel<|endoftext|>
Passage side: Body of the organism is made up of a single cell. The
              body of multicellular ...<|endoftext|>
```

The DenseRetrieval-then-Reranking task: Similar to the BM25-then-Reranking setup, I first build an initial candidate pool using the trained dense retrieval model (retrieving the top 1,000 passages for all training queries). Then, I sample N training queries, retaining all relevant passages as positives and sampling M negative passages from the remaining non-relevant candidates. The resulting data schema is exactly the same as that used in the BM25-then-Reranking pipeline.

Table 1: Performance comparison across baseline models (BM25, MonoBERT, TFR-BERT, RankT5-Large) and Qwen3-0.6B-Reranker based fine-tuned models in terms of MRR@10 and nDCG@10.

	Model Size	MRR@10	nDCG@10
BM25	-	0.1876	0.2340
MonoBERT-large [9]	0.34B	0.3666	0.4262
TFR-BERT Single Run [10]	0.34B	0.3782	-
RankT5-large [15]	0.77B	0.3851	0.4453
monoBERT + duoBERT (leaderboard best for non-ensemble) [9]	0.11B ($\times$2 models)	**0.3900**	-
TFR-BERT Ensemble (Leaderboard best ensemble) [10]	0.34B ($\times$15 models)	**0.4046**	-
Qwen3-0.6B-Reranker	0.60B	0.3289	0.3911
Qwen3-0.6B-Reranker-SFT (100K queries, w/o TCP)	0.60B	0.3817	0.4399

2.3 Experiment Setup

Implementation Details. In this report, all of my experiments are trained and evaluated on an 8×16GB NVIDIA Tesla V100 GPU cluster. Due to hardware resource constraints: (1) for the re-ranking task, I set the maximum sequence length to 384 and the effective batch size to 256; (2) for the retrieval task, I cap the maximum query and document lengths at 64 and 144 tokens, respectively, with effective batch sizes varying across different retrieval models. Following standard language model fine-tuning practices, I adopt the AdamW optimizer with a default learning rate of $1e{-}5$.

Evaluation Metrics. Both the retrieval and re-ranking tasks are evaluated using three metrics: Recall@K ($K \in \{100, 200, 1000\}$), MRR@10, and nDCG@10. Formal definitions of these metrics can be found in Han et al. [10]. In general, Recall primarily assesses corpus-wide retrieval coverage, whereas MRR and nDCG quantify top-rank precision for the re-ranking tasks.

3 Experimental Results

3.1 Reranking the official BM25 retrieval result (BM25-then-Reranking)

In this section, my primary focus is to re-rank BM25-retrieved results using models based on Qwen3-0.6B-Reranker. I first evaluate the zero-shot ranking performance of the original Qwen3-0.6B-Reranker (without fine-tuning). For a fairer comparison, I include fine-tuned MonoBERT [9] and fine-tuned RankT5-Large [15] as strong baselines. As shown in Table 1, the vanilla Qwen3-0.6B-Reranker achieves an MRR@10 of 0.3289, clearly demonstrating its capability to identify relevant passages beyond the initial BM25 ranking. Furthermore, even with a relatively small training dataset (a total of 100K queries, pairing each query with 1–2 relevant and 10 irrelevant passages), Supervised Fine-Tuning (SFT) yields a substantial performance boost, nearly matching the top-performing non-ensembled models on the leaderboard.

In the following sections, I detail further explorations aimed at improving model performance. As an additional reference, I also experiment with RankT5-3B, which achieves an 0.3997 in MRR@10 and an 0.4586 in nDCG@10. However, due to the GPU memory constraints, I could not conduct further experiments with the larger Qwen3-Reranker-4B/8B variants. Nevertheless, these results strongly suggest that ranking performance could be further scaled up alongside model parameter size.

Target Corpus Pre-training (TCP). Nogueira et al. [9] developed a multi-stage document re-ranking pipeline where the initial stage involves continued pre-training of BERT on the MS MARCO corpus. Inspired by this idea, I also conduct Target Corpus Pre-training (TCP), continuing language model pre-training directly on the MS MARCO dataset. Specifically, I

Table 2: Impact of Target Corpus Pre-training on fine-tuned model evaluated by MRR@10 and nDCG@10.

	SFT 10K	SFT 10K w/ TCP	SFT 50K	SFT 50K w/ TCP	SFT 100K	SFT 100K w/ TCP	SFT 200K	SFT 200K w/ TCP
MRR@10	0.3618	0.3626 (+0.22%)	0.3749	0.3771 (+0.59%)	0.3817	0.3832 (+0.39%)	0.3835	0.3846 (+0.29%)
nDCG@10	0.4204	0.4211 (+0.17%)	0.4338	0.4352 (+0.32%)	0.4399	0.4418 (+0.43%)	0.4433	0.4441 (+0.18%)

Table 3: Performance of fine-tuned Qwen3-0.6B-Reranker under varying training data scales. Here, M denotes the number of sampled queries, and K denotes the number of sampled irrelevant passages per query.

	Base (w/o SFT)	Model 1 (M=100K, K=10)	Model 2 (M=300K, K=10)	Model 3 (M=500K, K=10)	**Model 4 (M=500K, K=14)**	Model 5 (M=500K, K=18)
MRR@10	0.3289	0.3832	0.3859	0.3922	**0.3952**	0.3934
nDCG@10	0.3911	0.4418	0.4441	0.4509	**0.4537**	0.4534

perform TCP on the Qwen3-0.6B-Reranker model using a standard causal language modeling (CLM) objective—namely, next-token prediction loss. The training corpus is the MS MARCO passage collection, pre-tokenized offline into fixed-length chunks of 512 tokens. Consistent with Nogueira et al. [9], I observe a clear incremental performance gain when adopting TCP, and this performance boost remains consistent across different training data scales. Throughout this technical report, this TCP-adapted model serves as the backbone for all downstream fine-tuned rerankers.

Supervised Fine-tuning (SFT) with Varying Training Data Scales. To understand how the volume of training data affects fine-tuning performance, I experiment with training datasets of varying sizes. These subsets are created by sampling queries and their corresponding passages from the full corpus: specifically, I sample M queries, pairing each with N relevant and K irrelevant passages. In the experiments below, I evaluate $M \in \{100\text{K}, 300\text{K}, 500\text{K}\}$, with N=1 and $K \in \{10, 14, 18\}$. Notably, 500K queries cover nearly the entire corpus. As shown in Table 3 below, supervised fine-tuning significantly boosts the model's ranking ability, with performance exhibiting an upward trend as more training queries are introduced. In addition, I examine different numbers of negative samples (K=10, 14, and 18), revealing that K=14 yields the optimal performance, whereas a further increase in negative samples degrades the metrics. Notably, the MRR@10 of 0.3952 already outperforms the best non-ensemble baseline in Table 1.

Model Ensemble (ME). Beyond individual fine-tuning, I employ model ensembling to boost ranking performance [16]. Following the same sampling strategy (sampling M queries, each with N relevant and K irrelevant passages), I generate six distinct datasets and train six independent models using identical configurations. I then experiment with two ensemble strategies: prediction-level ensemble [16] via score fusion, and weight-space ensemble (the Model Soup algorithm [17]), which averages model weights across multiple runs. The resulting soup model, **BR1**, surpasses the strongest single-model baseline by 2.74%, and the prediction ensemble achieves performance on par with the TFR-BERT ensemble [10] using just six runs. These results validate both the utility of Qwen3-0.6B-Reranker for fine-tuning and the effectiveness of my ensemble pipeline.

Takeaway

Across the experiments above, two conclusions stand out: first, combining target corpus pre-training, data scaling, and model merging allows my single model to beat the previous best single-model baseline (0.4007 vs. 0.3900 MRR@10), even slightly surpassing the 3B RankT5 model (0.3997). Second, ensembling only a few models achieves performance on par with the best ensemble on the leaderboard (0.4050 vs. 0.4046 MRR@10).

Table 4: Performance summary of different approaches. Improvements are calculated relative to the average baseline performance.

	Average performance (of 6 models)	Model soup* [17] (on 3 models)	Result ensemble (on 3 models)	**BR1**: Model soup* [17] (on 6 models)	Result ensemble (on 6 models)
MRR@10	0.3952	0.3992 (+1.01%)	0.4019 (+1.70%)	**0.4007 (+1.39%)**	**0.4050 (+2.48%)**
nDCG@10	0.4537	0.4578 (+0.90%)	0.4600 (+1.39%)	**0.4592 (+1.21%)**	**0.4629 (+2.03%)**

Note: The model soup algorithm creates a single model with weights averaged from individual models; therefore, its inference resource consumption is equivalent to that of a single model.

3.2 Retrieving the whole data corpus (the Dense Retrieval task)

For a given query, the retrieval task retrieves the most relevant passages from the entire corpus (around 8.8 million passages). To ensure low inference latency, industrial systems commonly adopt the dual-encoder architecture [11, 12] that encodes queries and documents separately and computes their dot-product similarity for online inference. In this subsection, I adopt the dual-encoder as the default architecture, and using Qwen3-0.6B-Embedding as the backbone model. To assess the capability of this backbone and evaluate my subsequent improvements, I benchmark against various representative baselines, including DeepCT [25], GTR-Large [20], CoROM-Retriever [31], coCondenser [11], SPLADE-coCondenser [19], SimLM [21], CoT-MAE [26], and ColBERTv2 [22].

As shown in Table 5, even without supervised fine-tuning, the vanilla Qwen3-0.6B-Embedding model achieves a Recall@1000 of 0.9799 and an MRR@10 of 0.3165. Fine-tuning on the full training query set (pairing each query with 23 GTR-Large hard negatives) allows it to surpass GTR-Large across both MRR and nDCG metrics, demonstrating the clear efficacy of fine-tuning the Qwen3-0.6B-Embedding backbone. Among all baseline models, SimLM [21] and ColBERTv2 [22] yield the strongest performance. Specifically, SimLM introduces a replaced language modeling pre-training objective alongside a multi-stage pipeline featuring hard negative mining and cross-encoder distillation. Meanwhile, albeit at the expense of higher inference latency and storage footprint, ColBERTv2 adopts the late-interaction paradigm—independently encoding query and document tokens via BERT and aggregating relevance through MaxSim operators—which delivers the highest overall scores in Table 5. Finally, while prior work [6] showed that the LLaMA-7B representation model achieves state-of-the-art performance on both Recall@1000 (0.9940) and MRR@10 (0.4120), I did not evaluate it in this technical report due to GPU memory constraints.

Hard Negative Mining. According to Table 5, the vanilla Qwen3-0.6B-Embedding model does not deliver performance on par with other dense retrieval models. A primary reason is that those high-performing models were all thoroughly fine-tuned on the MS MARCO dataset. Therefore, I conduct a series of fine-tuning experiments in this section. For the retrieval task, hard negative mining plays a pivotal role [14, 19, 20]. Here, I experiment with several strategies to mine high-quality hard negatives, drawing from BM25, GTR-Large [20], SPLADE-coCondenser [19], SimLM [21], ColBERTv2 [22], and my best cross-encoder reranker (model **FR1** from Section 3.3).

More specifically, for each query, I use all relevant passages as positives, and randomly sample 23 non-relevant passages ranked between positions 10 and 200 (retrieved by a dense retriever or a cross-encoder reranker) as hard negatives. In-batch negatives (easy negatives) are shared across other queries within the same training batch. Training is carried out with 6 queries per batch per GPU across 8× NVIDIA V100 GPUs, resulting in an effective batch size of 1,152 passages per iteration. Under this setup, the ratio of easy negatives to hard negatives is approximately 49:1.

The retrieval results across different hard negative mining strategies are summarized in Table 6. Compared to the vanilla Qwen3-0.6B-Embedding baseline, mining hard negatives from stronger retrieval models (e.g., GTR-Large, SPLADE-coCondenser, SimLM, and ColBERTv2) consistently boosts MRR@10 and nDCG@10, while Recall@1000 remains relatively stable. Beyond standard dense retrievers, I also evaluate hard negatives mined by the lat-

Table 5: Performance comparison across BM25, Qwen3-0.6B-Embedding, and representative dense retrieval baselines in terms of Recall@1000, MRR@10, and nDCG@10.

Retrieval	Model Size	Recall@1000	MRR@10	nDCG@10
BM25 - official [1]	-	0.8140	0.1670	-
BM25 - optimized [13]	-	0.8690	0.1876	0.2340
DeepCT [25]	-	0.9100	0.2430	-
GTR-Large [20]	0.34B	**0.9873**	0.3623	0.4268
CoROM-Retriever [31]	0.11B	0.9799	0.3682	0.4323
coCondenser [11]	0.11B	0.9779	0.3705	0.4337
CoT-MAE [26]	0.11B	0.9805	0.3715	0.4350
SPLADE-CoCondenser [19]	0.34B	0.9825	0.3809	0.4471
SimLM [21]	0.34B	0.9819	0.3947	0.4585
ColBERTv2 (late interaction) [22]	0.11B	0.9738	**0.3968**	**0.4629**
Qwen3-0.6B-Embedding	0.60B	0.9799	0.3165	0.3796
Qwen3-0.6B-Embedding (w/ 23 GTR-Large hard negatives)	0.60B	0.9801	0.3789	0.4437

Note: To ensure fair and consistent evaluation (eliminating confounding factors from data preprocessing and metric computation pipelines), all baselines — including GTR-Large, coCondenser, SPLADE-coCondenser, SimLM, CoT-MAE, and ColBERTv2 — are evaluated locally under an identical protocol. Due to an input discrepancy (evaluations in [21, 26] incorporate passage title content, whereas our setup does not), the reproduced MRR@10 is 0.3947 for SimLM (vs. 0.4110 reported in [21]) and 0.3715 for CoT-MAE (vs. 0.3940 reported in [26]). Similarly, evaluating the official GTR-Large checkpoint from Hugging Face [24] yields comparable nDCG@10 but a 1.7 pp drop in MRR@10 compared to the number reported in [20]. This impact of title discrepancy was also highlighted by Lassance and Clinchant [23].

Table 6: Performance comparison of different hard negative mining strategies in terms of Recall@1000, MRR@10, and nDCG@10.

Retrieval	Recall@1000	MRR@10	nDCG@10
Qwen3-0.6B-Embedding	0.9799	0.3165	0.3796
Qwen3-0.6B-Embedding (w/ 23 GTR-Large hard negatives)	0.9801	0.3789	0.4437
Qwen3-0.6B-Embedding (w/ 23 SPLADE-CoCondenser hard negatives)	0.9763	0.3777	0.4416
Qwen3-0.6B-Embedding (w/ 23 SimLM hard negatives)	0.9803	0.3783	0.4436
Qwen3-0.6B-Embedding (w/ 23 ColBERT V2 hard negatives)	0.9790	0.3784	0.4431
Qwen3-0.6B-Embedding (w/ 23 cross-encoder hard negatives)	**0.9827**	**0.3853**	**0.4501**

ter cross-encoder reranker **FR1**, which achieves the highest Recall@1000, MRR@10, and nDCG@10 scores across all mining strategies.

Model Ensemble (ME). Similar to the approach taken for the BM25 re-ranking model **BR1**, I employ model ensembling to further improve dense retrieval performance. Following the same sampling strategy used for cross-encoder hard negative mining (i.e., retaining all relevant passages as positives and randomly sampling 23 hard negatives per query), I generate six distinct datasets and train six independent models with identical configurations. I then apply the Model Soup algorithm [17] by averaging the weights across all six models into a single checkpoint, denoted as **DR1**.

Retrieval results are presented in Table 7. Compared to GTR-Large, the soup model **DR1** achieves significantly higher MRR@10, nDCG@1000, Recall@100, and Recall@200, while maintaining a comparable Recall@1000 with only a marginal difference. Compared to ColBERTv2, **DR1** attains superior Recall@100, Recall@200, and Recall@1000 at the expense of slightly lower MRR and nDCG. Notably, **DR1** stands as the top-performing dual-encoder retrieval model among all evaluated baselines. Additionally, I experimented with combining the model soup strategy with MaxSim token-level late interaction (as done in ColBERTv2); the resulting model, **DR2**, delivers the highest MRR, nDCG, and Recall scores overall. However, to preserve architectural simplicity and minimize inference latency, DR1 remains my default dense retrieval backbone for the full ranking pipeline.

Table 7: Performance comparison of different model ensembling strategies for dense retrieval across Recall@K, MRR@10, and nDCG@10.

Retrieval	Recall@100	Recall@200	Recall@1000	MRR@10	nDCG@10
GTR-Large [20]	0.9084	0.9494	**0.9873**	0.3623	0.4268
ColBERTv2 (late interaction) [22]	0.9135	0.9455	0.9738	0.3968	0.4629
Qwen3-0.6B-Embedding (w/ 23 cross-encoder hard negatives)	0.9083	0.9446	0.9827	0.3853	0.4501
DR1: Qwen3-0.6B-Embedding model soup (6 models $\rightarrow$ 1 model)	0.9172	0.9528	0.9861	0.3952	0.4614
DR2: Qwen3-0.6B-Embedding model soup + MaxSim (as in [22])	**0.9188**	**0.9521**	0.9855	**0.3968**	**0.4630**

Takeaway

In this subsection, I have explored multiple model optimization techniques—primarily hard negative mining and model ensembling—to enhance dense retrieval performance. Through extensive experimentation, fine-tuned retrievers built upon the Qwen3-0.6B-Embedding model, even without specialized pre-training or architectural modifications, achieve near-SOTA performance across both Recall@1000 (0.9861 from **DR1** vs. 0.9873 from the similarly sized GTR-Large) and MRR@10 (0.3952 from **DR1** vs. 0.3968 from ColBERTv2). Moreover, among all evaluated dual-encoder retrievers (excluding ColBERTv2 and **DR2**, which adopt the late-interaction structure), the souped models **DR1** stand out as the top-performing systems. Finally, while I did not evaluate larger models such as GTR-XL, GTR-XXL, and RepLLaMA-7B due to GPU resource constraints, I believe these empirical findings and core conclusions remain robust.

3.3 Reranking the dense retrieval results (the DS-then-Reranking task)

For most of the dense retrieval models discussed above, Recall@1000 already exceeds 0.9800. Therefore, in this section, I focus primarily on re-ranking the top 1,000 candidate passages retrieved by the dense retriever—that is, the full ranking task. To rigorously assess subsequent model improvements, I benchmark against several representative reranker baselines, including ME-BERT [28], RankT5-large [15], CoROM [31], CoT-MAE [26], Ensemble of TFR-BERTs [10], RocketQA v2 [27], and HLATR [29]. For a fair comparison, the results of existing models in Table 8 are obtained by re-evaluating their publicly released checkpoints under my unified evaluation protocol. Entries marked with question marks denote models that could not be fully reproduced, either due to the unavailability of public checkpoints or ambiguities in the original input data (as noted by Lassance and Clinchant [23], some studies utilize passage titles whereas others do not).

I first evaluate the best BM25-then-Reranking model, BR1, on the top 1,000 candidates retrieved by DR1; the corresponding results are presented in Table 8. Compared to the fully reproduced baselines such as RankT5-large, CoROM, monoT5-large, and CoT-MAE Reranker, BR1 achieves highly competitive performance, attaining an MRR@10 of 0.4168 and an nDCG@10 of 0.4840. Although models like RocketQA v2, HLATR, and the ensemble of fine-tuned ERNIE and RoBERTa report higher MRR@10 scores, their results cannot be faithfully reproduced because their checkpoints are unavailable and their input configurations remain uncertain.

Training-Inference Alignment. The model BR1 was trained on 14 randomly sampled non-relevant passages from BM25 retrieval results, which does not closely reflect the candidate distribution produced by dense retrievers. This discrepancy can lead to a training-inference misalignment problem and degrade downstream ranking quality. To address this issue, I adopt a sampling procedure similar to that of BR1: I retain all labeled relevant passages and randomly sample 14 non-relevant candidates from the top 1,000 dense retrieval outputs. Using this newly constructed dataset, I then train a DenseRetrieval-then-Reranking model. Table 9 presents the corresponding performance. Clearly, a reranker trained directly on dense retrieval candidates better aligns with the dense retriever's output distribution, allowing the DR-then-Reranking model to outperform the BM25-then-Reranking counterpart (improving MRR@10 from 0.4087 to 0.4178).

Table 8: Performance comparison of full ranking models in terms of MRR@10 and nDCG@10. Entries marked with question marks indicate results that could not be fully reproduced, either due to the lack of public checkpoints or ambiguities in the evaluation data setup (e.g., whether passage titles were included [23]).

Retrieval	Reranker	Reranker Model Size	MRR@10	nDCG@10
BM25 - official [1]	-	-	0.1670	-
BM25 - optimized [13]	-	-	0.1876	0.2340
ME-BERT [28]	ME-BERT [28]	0.34B	0.3950?	?
GTR-Large [20]	RankT5-large [15]	0.77B	0.4047	0.4739
coCondenser [11]	CoROM [31]	0.11B	0.4136	0.4793
GTR-Large [20]	monoT5-large [32]	0.77B	0.4156	0.4843
RocketQA V2 [27]	RocketQA V2 [27]	0.12B	0.4190?	?
CoT-MAE Retriever [26]	CoT-MAE Reranker [26]	0.34B	0.4200	0.4853
DeepCT [25]	Ensemble of TFR-BERTs [10]	0.34B (×15 models)	0.4213	-
coCondenser [11]	HLATR (three-stage reranking) [29]	0.34B	0.4370?	?
Condenser [11]	Ensemble of finetuned ERNIE and RoBERTa	0.34B (×? models)	0.4430?	?
Best Qwen3 Retriever **DR1**	Best BM25-then-Reranking Model **BR1**	0.60B	0.4168	0.4840

Note: To ensure a fair and consistent evaluation, baseline models such as RankT5-large, CoROM, monoT5-large and CoT-MAE are evaluated locally under an identical protocol. Due to input discrepancies (e.g., the exclusion of passage titles in my setup) [23], the reproduced MRR@10 for CoT-MAE is 0.4200 (vs. 0.4388 reported in [26]), whereas the reproduced MRR@10 for CoROM matches that reported in [31]. For monoT5-large, the reproduced MRR@10 is 0.4156, while prior literature does not report results for monoT5-large paired with a GTR-Large retriever.

Table 9: A comparison of multiple fine-tuned full ranking models using MRR@10 and nDCG@10.

Reranker	Model Config	MRR@10	nDCG@10
Single BM25-then-Reranking Model	Randomly sample 14 negatives from BM25 results	0.4087	0.4756
Single DR-then-Reranking Model	Randomly sample 14 negatives from DR1 results	0.4178	0.4857
BR1: BM25-then-Reranking Model Soup	Weighted soup of six models (single model)	0.4168	0.4840
FR1: DR-then-Reranking Model Soup	Weighted soup of six models (single model)	0.4237	0.4909
FR2: FR1 + Retriever Score	Retriever score + reranker score	**0.4269**	**0.4943**
FR3: FR2 + HLATR (as done in [29])	Retriever score + reranker score + HLATR	**0.4296**	**0.4966**

Model Ensemble (ME). As with BR1, I apply the Model Soup algorithm to further enhance full ranking performance. Specifically, under identical training configurations, I repeat the training run across six distinct random seeds to produce six independent models. Averaging their weights via Model Soup yields model **FR1**, which achieves substantial gains (boosting MRR@10 from 0.4168 to 0.4237). Furthermore, since DR1 already delivers strong first-stage retrieval quality, I construct **FR2** by linearly combining the retriever score from DR1 with the reranker score from FR1, which yields additional gains across ranking metrics (lifting MRR@10 from 0.4237 to 0.4269). Prior work on HLATR [29] showed that incorporating a multi-stage reranking framework further enhances ranking quality. Following this intuition, I integrate an additional HLATR reranker on top of FR2, achieving an additional performance gain (increasing MRR@10 from 0.4269 to 0.4296). Ultimately, my final model outperforms all fully reproduced baselines in Table 8, nearly reaching an MRR@10 of 0.4300.

Takeaway

In this subsection, I focused on the full ranking task. Through extensive experimentation, three clear conclusions emerge: first, aligning the training and inference data distributions is crucial for reranker training—shifting the training negative pool from BM25 candidates to dense retrieval results yields a significant performance boost; second, applying Model Soup across multiple repeated training runs remains a highly effective strategy for boosting model performance; and third, ranking quality can be further amplified by combining retriever and reranker scores alongside a listwise post-reranker like HLATR. Notably, models FR1 through FR3 even outperform much larger 3B models such as RankT5-3B (which reports an MRR@10 of 0.4208), underscoring the strong parameter efficiency of my approach.

4 Conclusions

In this report, I investigated passage retrieval and re-ranking on the MS MARCO benchmark using Qwen3-0.6B-based models. Across three evaluation settings—BM25-then-reranking, corpus-wide dense retrieval, and dense-retrieval-then-reranking—the results demonstrate that competitive ranking performance can be achieved without complex architectural modifications, task-specific pre-training objectives, cross-encoder knowledge distillation, or customized loss functions. Instead, the primary performance gains stem from continued pre-training on the target corpus, rigorous training data curation, hard-negative mining, training-inference distribution alignment, and weight-space model merging via Model Soup [17]. In particular, Model Soup proves to be an exceptionally effective and cost-efficient strategy for boosting ranking quality while keeping inference latency strictly identical to that of a single model. Although evaluating larger models (such as the 4B and 8B variants of Qwen3-Embedding and Qwen3-Reranker) is left for future exploration, the present findings strongly suggest that further gains can be achieved by scaling model parameters in tandem with the data-centric and merging strategies explored here.

Another important finding concerns benchmark evaluation comparability. Existing MS MARCO results are often not directly comparable because certain prior works incorporate passage titles while others omit them. In my experiments, prepending passage titles consistently improves retrieval MRR@10 by approximately 2–3 percentage points and re-ranking MRR@10 by roughly 1–2 percentage points. Therefore, to ensure a fair comparison, I reproduced all major baselines using their publicly available checkpoints under a unified evaluation protocol. This observation is consistent with prior analyses by Lassance and Clinchant [23], highlighting the pressing need for transparent reporting of input preprocessing pipelines in information retrieval benchmarking.

Finally, this project offers an interesting case study in AI-assisted research. Throughout this work, I collaborated with Claude Code, powered by Claude Opus 4.7. The assistant proved highly effective in engineering-intensive tasks, including environment setup, model implementation, code debugging, data processing, experiment orchestration, and result analysis. It was particularly instrumental in diagnosing discrepancies between published and reproduced metrics by helping identify the inconsistent use of passage titles and designing targeted validation tests. However, the model frequently defaulted to generic solutions and exhibited limited research taste. Many of the most critical breakthroughs—such as applying Model Soup, aligning the reranker training distribution with dense retrieval outputs, and prioritizing data curation over architectural modifications—originated from human intuition rather than the assistant's suggestions. This aligns with recent observations that AI agents can dramatically accelerate engineering execution but still struggle with high-level research ideation and creative direction [8].

References

[1] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg. MS MARCO: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.

[2] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1, pp. 4171–4186, 2019.

[3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[4] K. Clark, M.-T. Luong, Q. V. Le, C. D. Manning. ELECTRA: Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555*, 2020.

[5] R. Pradeep, S. Sharifymoghaddam, J. Lin. RankZephyr: Effective and robust zero-shot listwise reranking is a breeze! *arXiv preprint arXiv:2312.02724*, 2023.

[6] X. Ma, L. Wang, N. Yang, F. Wei, J. Lin. Fine-tuning LLaMA for multi-stage text retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2421–2425, 2024.

[7] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.

[8] P. Kirgis, S. Kapoor, A. Schwartz, S. Rabanser, D. Africa, K. Voudouris, V. Nguyen, T. Pilditch, M. Dubois, H. Coppock, C. Ududec, N. Nadgir, M. Orona, T. Bayer, D. Chan-Sew, Y. Ling, A. Shetty, H. Toner, G. Hadfield, S. Lazar, S. Newman, S. Tekofsky, R. Bommasani, A. Narayanan. Can AI agents conduct open-ended AI research? Early evidence from two case studies. 2026.

[9] R. Nogueira, W. Yang, K. Cho, J. Lin. Multi-stage document ranking with BERT. *arXiv preprint arXiv:1910.14424*, 2019.

[10] S. Han, X. Wang, M. Bendersky, M. Najork. Learning-to-rank with BERT in TF-Ranking. Google AI Tech Report, 2020.

[11] L. Gao, J. Callan. Unsupervised corpus aware language model pre-training for dense passage retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pp. 2843–2853, 2022.

[12] Y. Qu, Y. Ding, J. Liu, K. Liu, R. Ren, W. X. Zhao, D. Dong, H. Wu, H. Wang. RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5835–5847, 2021.

[13] Pyserini. Pyserini: BM25 baseline for MS MARCO Passage Ranking. `https://github.com/castorini/pyserini/blob/master/docs/experiments-msmarco-passage.md`

[14] O. Khattab, M. Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48, 2020.

[15] H. Zhuang, Z. Qin, R. Jagerman, K. Hui, J. Ma, J. Lu, J. Ni, X. Wang, M. Bendersky. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2308–2313, 2023.

[16] Z. Chen, X. Lu, J. Li, P. Chen, Z. Li, K. Sun, Y. Luo, Q. Mao, M. Li, L. Xiao, D. Yang. Harnessing multiple large language models: A survey on LLM ensemble. *arXiv preprint arXiv:2502.18036*, 2025.

[17] M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, L. Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pp. 23965–23998. PMLR, 2022.

[18] OpenMatch — THU-MSR. BM25 + Electra Large. `https://github.com/thunlp/OpenMatch`

[19] T. Formal, C. Lassance, B. Piwowarski, S. Clinchant. From distillation to hard negative sampling: Making sparse neural IR models more effective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2353–2359, 2022.

[20] J. Ni, C. Qu, J. Lu, Z. Dai, G. Hernández Ábrego, V. Y. Zhao, Y. Luan, K. B. Hall, M.-W. Chang, Y. Yang. Large dual encoders are generalizable retrievers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 9844–9855. Association for Computational Linguistics, 2022.

[21] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, F. Wei. SimLM: Pre-training with representation bottleneck for dense passage retrieval. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pp. 2244–2258, 2023.

[22] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 3715–3734, 2022.

[23] C. Lassance, S. Clinchant. The tale of two MSMARCO—and their unfair comparisons. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2431–2435, 2023.

[24] sentence-transformers. gtr-t5-large. https://huggingface.co/sentence-transformers/gtr-t5-large

[25] Z. Dai, J. Callan. Context-aware sentence/passage term importance estimation for first stage retrieval. *arXiv preprint arXiv:1910.10687*, 2019.

[26] X. Wu, G. Ma, M. Lin, Z. Lin, Z. Wang, S. Hu. Contextual masked auto-encoder for dense passage retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, pp. 4738–4746, 2023.

[27] R. Ren, Y. Qu, J. Liu, W. X. Zhao, Q. She, H. Wu, H. Wang, J.-R. Wen. RocketQAv2: A joint training method for dense passage retrieval and passage re-ranking. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2825–2835, 2021.

[28] Y. Luan, J. Eisenstein, K. Toutanova, M. Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.

[29] Y. Zhang, D. Long, G. Xu, P. Xie. HLATR: Enhance multi-stage text retrieval with hybrid list aware transformer reranking. *arXiv preprint arXiv:2205.10569*, 2022.

[30] D. Long, Y. Zhang, G. Xu, P. Xie. Retrieval oriented masking pre-training language model for dense passage retrieval. *arXiv preprint arXiv:2210.15133*, 2022.

[31] CoROM. CoROM semantic relevance — English — general domain — base. https://modelscope.cn/models/iic/nlp_corom_passage-ranking_english-base

[32] R. Pradeep, R. Nogueira, J. Lin. The expando-mono-duo design pattern for text ranking with pretrained sequence-to-sequence models. *arXiv preprint arXiv:2101.05667*, 2021.